

SFT initialization: start with plain, long CoT, and then additional
⇒ claimed benefit: interpretability ⇒ generate well-formatted verification
⇒ Note: origins on the data not quite clear... CoT.

SFT for Reasoning/math:

⇒ for Reasoning/math ability from LLMs. A small number of samples for bootstrapping Reasoning
non-verifiable rewards

RL steps: RL part is same for dPSK-R1-zero and dPSK-R1
Minor difference: additional language consistency ~~less~~ rewards
for longer CoT. interesting: RL ~~naturally~~ leads to mixed languages
Cons Sometimes

SFT/RLHF: the usual post-training process happens after RLVR/
Reasoning RL

for alignment

SFT step, 2 epochs:

dPSK: post-training

- Reasoning data → non-verifiable tasks, use V3 as a judge
- non-Reasoning data - V3 SFT dataset (200K)

RLHF step: alignment

- Re-use R1-zero style reasoning RLHF
- non-verifiable tasks - V3 RLHF pipeline, still use GRPO for RLHF

How well does R1 work? : pretty good

Distillation: can we get non-reasoning models to reason data
pipeline: ①: Have R1 generate CoT traces (800K)
②: teach Qwen 2.5 via distillation SFT

insights for supervised distillation:

- ①: small language model can obtain significant reasoning ability
- ②: supervised distillation is better than RL for post-training small language model

③: open-source: dpo-R₁ → 800k CoT trace data → SFT
Gwen2.5

{ coding
math
scientific discovery } ← Reasoning-Gwen2.5 ←

Kimi-K1.5: why do we study this?

- ①: Released at the same time as R₁
- ②: also beats ol using RL
- ③: scaling Reinforcement Learning with LLMs
- ④: Technical Report for KIMI-1.5

pros: Long CoT Reasoning strategy: key steps: filtering

- ①: Dataset Construction (difficulty ~~conservation~~)
- ②: SFT: long CoT
- ③: RL: with their own policy gradient loss

↓ data/curation/SFT: data
math-style setting, blanking topics
{ exclude multiple choice / True False
select only examples that models fail on
best-of-8

1. DPO-type derivation for RL
{ Baseline policy gradient w/ regularization = like GRPO clipping
Reference-based Reward model